DEVELOPER TOOLING

JHipster Entity Generator

Entity generation from JDL in seconds, safe to re-run

Java · JHipster · JDL · ~4 hours to ~2 seconds

Shivam Paliwal
Solutions Engineer · paliwalshivam93.vns@gmail.com

Technical documentation · October 2026

Contents

1. Overview 3

Results . . . 3

At a glance . . . 3

2. The problem it solves 3

What one entity involves in a JHipster application . . . 3

Two recurring costs . . . 4

3. The solution 4

Fast generation from JDL . . . 4

Safe regeneration . . . 4

4. Workflow 5

Day-to-day steps . . . 5

Example JDL model . . . 5

Evolving the model . . . 5

5. Comparison 6

6. Good practice 6

Works well with the other libraries . . . 6

7. Availability 7

1. Overview

JHipster Entity Generator is a developer tool that generates application entities directly from a JHipster Domain Language (JDL) model. Work that used to take around four hours of manual effort now finishes in about two seconds, and running it again after the model changes does not overwrite changes developers have already made.

Results

Measure	Before	With the generator
Time to create entities from a model	~4 hours of manual work	~2 seconds
Re-generating after a JDL change	Custom changes are overwritten or must be merged by hand	Existing custom changes are preserved
Consistency across entities	Depends on who wrote the code	Every entity follows the same generated structure

At a glance

Item	Value
Type	Internal developer productivity tool
Input	A JHipster JDL file describing entities, fields, validations and relationships
Output	Entity code for a JHipster application
Stack	Java, JHipster, JDL
Key properties	Seconds instead of hours; safe to run repeatedly
Availability	Internal. The source repository is not public.

2. The problem it solves

What one entity involves in a JHipster application

In a JHipster application, an entity is much more than one class. With typical options (DTOs with MapStruct, a service layer, filtering and pagination), each entity spans every layer of the stack:

Layer	Typical artifacts
Domain and persistence	JPA entity, Spring Data repository, Liquibase changelog for the table and its constraints, sample data

Layer	Typical artifacts
Service	Service interface and implementation, DTO, MapStruct mapper, criteria class and query service for filtering
REST API	REST resource with create, update, partial update, list with paging and filters, get and delete
Front end	Model, HTTP service, list, detail, create/edit and delete screens, routes, translation files
Tests	Entity, DTO and mapper unit tests, REST integration tests

That is typically more than twenty files per entity, and a business module usually has many entities with relationships between them.

Two recurring costs

- **Creation effort.** Producing and wiring all of these artifacts by hand, and adapting them to project conventions, took around four hours for a set of entities. The work is repetitive, so mistakes are easy: a missed constraint, a wrong column type, a mapper field left out.
- **Regeneration risk.** Models change: fields get added, validations tighten, relationships appear. Generated code is then customized with business rules, extra queries and UI changes. With the standard approach, regenerating after a JDL change asks to overwrite each modified file, or overwrites them all when forced. Teams either redo their customizations or merge every file by hand, so many simply stop regenerating and the JDL drifts from the code.

3. The solution

Fast generation from JDL

The generator reads the JDL model and produces the entity code in about **two seconds**. The JDL file becomes the single source of truth for the data model. A new entity, field or relationship is a few lines of JDL followed by one run of the generator.

Safe regeneration

The defining property of the tool: when the generator runs a **second time** on an updated JDL, it does **not change the code that was already changed** after the first generation. Developers can customize generated code freely and keep evolving the model, without losing work and without merging file by file.

> **What this means in practice.** You can add a field to the JDL six months after go-live and regenerate. The service logic, custom queries and UI changes your team added in the meantime stay as they are.

4. Workflow

Day-to-day steps

1. **Model:** describe or update entities, fields, validations and relationships in the JDL file.
2. **Generate:** run the generator against the JDL. It completes in about two seconds.
3. **Review:** look at the diff. New entities and fields appear, and earlier customizations are untouched.
4. **Customize:** add business logic to the generated code as usual.
5. **Verify and commit:** build, run the tests, and commit the JDL together with the generated code.

Example JDL model

A small model in standard JDL syntax, of the kind the generator takes as input:

```
entity Department {
  name String required maxlength(100)
  code String required unique
}

entity Employee {
  firstName String required
  lastName  String required
  email     String required unique
  isEnabled Boolean
}

relationship ManyToOne {
  Employee{department(name)} to Department
}

dto * with mapstruct
service * with serviceImpl
paginate * with pagination
filter *
```

Evolving the model

Later the business asks for a joining date on employees. The change is one line of JDL:

```
entity Employee {
  firstName   String required
  lastName    String required
  email       String required unique
  isEnabled   Boolean
  joiningDate LocalDate        // new
}
```

Running the generator again adds the new field. Custom code that was added to the employee classes and screens after the first run stays as it was.

5. Comparison

	Writing by hand	Standard regeneration	JHipster Entity Generator
Time per run	Hours (~4 h)	Minutes, plus resolving conflicts	~2 seconds
Consistent structure	Varies by developer	Yes	Yes
Re-run after a JDL change	Edit every affected file	Overwrite prompts for modified files	Runs again safely
Custom changes on re-run	Not applicable	Lost or merged manually	Preserved
JDL stays in sync with code	Rarely	Often abandoned after customization	Yes, it remains the source of truth

6. Good practice

- Keep the JDL file in version control next to the code, and change the model only through it.
- Make one model change per commit, so each regeneration is easy to review.
- Always review the generated diff before committing. The generator is fast, but a review catches model mistakes.
- Run the build and the test suite after every generation, including the database migration.
- Treat the JDL as living documentation of the data model, and use it in design discussions.

Works well with the other libraries

- **Duplicate Check Annotation:** add `@CheckDuplicateEntity` rules to generated entities to enforce business uniqueness on every save. They are custom changes, so regeneration keeps them.
- **Sticky Notes Library:** its REST contract uses JHipster-style criteria filters, so a Note entity generated from JDL can serve it, and the component can be dropped into any generated screen.

7. Availability

The generator is an internal tool and its source code is not published. For a demonstration, access or questions, contact the author:

Shivam Paliwal, Solutions Engineer · paliwalshivam93.vns@gmail.com